

AWS Well-Architected Framework Review — ROCA Event Management Platform (Batch 2 of 4)

1 Executive Summary

This Batch 2 Well-Architected review of ROCA's Event Creation and Event Discovery & RSVP flows reveals that five unresolved Batch 1 gaps — missing WAF, DLQs, Step Functions, EventBridge, and JWT rotation — now compound into critical risks as Stripe payment processing and high-concurrency RSVP logic enter scope. Sixteen recommendations across all six pillars are prioritized into three tiers, with five P0 items that should be treated as blockers before any production payment processing.

The most urgent finding is that ROCA's PCI DSS posture is incomplete. While Stripe Checkout qualifies the platform for SAQ-A (the simplest self-assessment), updated PCI SSC eligibility criteria effective March 2025 require Content Security Policy headers and script-integrity controls on all pages hosting Stripe Elements [1, 2]. The absence of WAF on API Gateway payment endpoints [3, 4] and GuardDuty for continuous threat monitoring [5] leaves the platform exposed to both compliance failure and active attack vectors.

Second, RSVP capacity enforcement requires a dual-layer architecture to prevent overselling: a Redis Lua script provides atomic, sub-millisecond admission gating [6], while a PostgreSQL conditional UPDATE (`remaining - 1 WHERE remaining > 0`) serves as the transactional source of truth [7]. Compensation logic must reconcile Redis state on failed database commits [8], and waitlist promotion should flow through SQS FIFO queues with Lambda reserved concurrency to protect downstream resources [9, 10].

Third, cancellation cascades for recurring events and RRULE reliability demand the Step Functions orchestration identified as missing in Batch 1. Saga-pattern workflows with compensating transactions [11, 12] are essential for coordinating refunds, RSVP state changes, and notifications across series cancellations, while RRULE validation must enforce bounded rules and IANA timezone expansion to prevent silent drift at DST transitions [13, 14]. Organizations should resolve all P0 and P1 items within 60 days to establish the architectural foundation required for Batch 3 and Batch 4 flows.

2 Introduction and Scope

This report presents the second batch of the AWS Well-Architected Framework Review for the ROCA Event Management Platform, covering the **Event Creation** and **Event Discovery & RSVP** flows. Event Creation encompasses a 6-step wizard supporting RRULE recurring patterns, Stripe ticketing and payment integration, volunteer role assignment, and a DRAFT→PUBLISHED→ACTIVE→COMPLETED lifecycle. Event Discovery & RSVP includes browse/search functionality, RSVP processing with capacity management and waitlist logic, Stripe checkout, and a PENDING→CONFIRMED→CHECKED_IN→ATTENDED state machine.

The architecture under review comprises AWS Lambda, API Gateway REST, PostgreSQL 15 on RDS Multi-AZ, Redis 7 (ElastiCache), S3/CloudFront for static assets, SES/SNS for notifications, Stripe for payment processing, WebSocket for real-time updates, and JWT/OAuth 2.0 for authentication. Batch 1 of this review identified fifteen gaps across all six pillars, with five critical findings that carry forward and compound in Batch 2: missing Step Functions orchestration, absent JWT token rotation, no Dead Letter Queues (DLQs), no WAF or GuardDuty deployment, and no EventBridge integration [15]. These gaps are particularly consequential for Batch 2 because the Event Creation and RSVP flows introduce payment processing (triggering PCI DSS scope), high-concurrency reservation logic (requiring atomic capacity enforcement), and multi-step cancellation cascades (demanding orchestrated saga patterns). This review evaluates all six Well-Architected pillars with focused analysis on PCI DSS compliance, RSVP race conditions, capacity enforcement, RRULE reliability, search performance, and cancellation cascades.

3 Security and PCI DSS Compliance

PCI DSS v4.0.1 SAQ-A Eligibility. ROCA's use of Stripe Checkout (redirect) and Stripe Elements (iframe) with Lambda backends qualifies the platform for SAQ-A, the simplest self-assessment questionnaire, because Stripe as a PCI DSS Level 1 certified service provider handles all cardholder data directly [16]. However, the PCI Security Standards Council updated SAQ-A eligibility criteria effective March 31, 2025, removing Requirements 6.4.3, 11.6.1, and 12.3.1 and replacing them with two new eligibility conditions: payment page elements must originate from a PCI DSS-compliant Third-Party Service Provider (TPSP) with an Attestation of Compliance on file, and merchants must confirm their entire site is not susceptible to malicious script attacks that could affect the TPSP payment elements [1, 2]. Merchants who cannot meet the script-security eligibility criteria must instead use SAQ A-EP, which includes significantly more requirements [17]. ROCA must therefore implement Content Security Policy headers and Subresource Integrity checks on all pages hosting Stripe Elements to maintain SAQ-A eligibility.

Critical Gap: WAF and GuardDuty. The Batch 1 finding of missing WAF and GuardDuty is now elevated to critical severity given the payment processing scope. AWS WAF on API Gateway protects REST APIs from SQL injection and cross-site scripting attacks, directly satisfying PCI DSS Requirement 6.4 for web application firewall protection [3]. AWS announced WAF support for PCI DSS 4.0 compliance protection with partner solutions in March 2025 [4]. GuardDuty addresses Requirement 11 by providing continuous threat detection monitoring of CloudTrail events and VPC Flow Logs [5]. CloudTrail satisfies Requirement 10 for logging and monitoring all access to cardholder data environments [18]. Lambda functions handling payment metadata should use KMS-encrypted environment variables with VPC endpoints for additional protection, satisfying Requirement 3 for encryption at rest [19]. All of these AWS services — Lambda, API Gateway, WAF, CloudTrail, KMS, GuardDuty, and VPC — are within scope for AWS’ s own PCI DSS compliance certification [20].

Stripe Webhook Security. Webhook endpoints must verify the Stripe-Signature header using HMAC-SHA256 by calling `constructEvent()` with the raw request body, the signature header, and the endpoint secret [21]. Best practices include using the raw (unparsed) request body for verification, validating the timestamp component to prevent replay attacks, storing the webhook signing secret in KMS or AWS Secrets Manager rather than plaintext environment variables, and implementing idempotent event processing to handle duplicate deliveries [22]. The Batch 1 JWT rotation gap also carries forward: in serverless architectures, organizations remain fully responsible for securing application code, data handling, and access controls under the shared responsibility model [23].

4 Reliability — RSVP Race Conditions and Capacity Enforcement

High-concurrency RSVP flows present the classic overselling problem: multiple simultaneous requests can read the same capacity count and all succeed, exceeding the event’ s actual capacity. ROCA requires a dual-layer enforcement pattern that combines Redis as a fast admission gate with PostgreSQL as the authoritative source of truth.

Layer 1: Redis Lua Atomic Fast-Path. Redis guarantees that Lua scripts execute atomically — no other command runs between script start and end — making them ideal for check-and-decrement capacity logic that rejects requests before they reach the database [6]. The canonical Lua pattern checks the current count and decrements only if capacity remains: if the returned value is zero or positive, the request proceeds to Layer 2; if negative, the request is immediately rejected or routed to the waitlist. Redis’ s WATCH/MULTI/EXEC optimistic locking provides an alternative for simpler inventory flows [24].

Layer 2: PostgreSQL Conditional UPDATE. The recommended SQL pattern is `UPDATE events SET remaining = remaining - 1 WHERE id = \ $1 AND remaining > 0 RETURNING remaining`, which atomically decrements capacity and fails gracefully when sold out [7]. For seat-specific assignment, `SELECT FOR UPDATE SKIP LOCKED` allows concurrent workers to skip already-locked rows and claim the next available slot, eliminating contention and deadlocks [7]. Advisory locks via `pg_try_advisory_lock(event_id)` provide an alternative application-level mutex that avoids row-lock overhead when serializing all reservations for a single event [7]. AWS RDS documentation confirms that `Lock:tuple wait` events spike when multiple sessions contend on the same row, reinforcing the need for `SKIP LOCKED` or advisory locks to reduce hot-row contention [25].

Compensation and Consistency. AWS recommends cache-aside or write-through patterns for keeping Redis and the database consistent [8]. If the PostgreSQL commit fails after Redis has already decremented, a compensation step must increment the Redis counter back (`INCR`) to restore accurate capacity state. This dual-layer approach ensures that the vast majority of over-capacity requests are rejected at the Redis layer (sub-millisecond), while PostgreSQL provides the transactional guarantee of correctness.

Chart Explanation: This diagram illustrates the two-layer capacity enforcement pattern. RSVP requests first pass through a Redis Lua script that atomically checks and decrements capacity [6]. Requests that pass proceed to PostgreSQL for a conditional `UPDATE` as the source of truth [7]. Failed PostgreSQL commits trigger a compensation `INCR` back to Redis [8]. Exhausted-capacity requests are routed to an SQS FIFO waitlist queue with DLQ safety [10], processed by Lambda with reserved concurrency to protect the database [9].

Waitlist Promotion. For waitlist management, an SQS FIFO queue ensures ordered processing of promotion candidates. Lambda reserved concurrency caps parallel executions to prevent overwhelming the database during bulk promotions [9]. Failed promotions route to an SQS dead-letter queue after a configurable `maxReceiveCount`, preserving messages for retry or manual intervention [10]. Lambda handlers must be idempotent since SQS may redeliver messages on failure [26].

5 Reliability — RRULE Recurring Events and Cancellation Cascades

Materialization Strategy. The recommended approach is a hybrid schema with lazy materialization: store a single master event row containing the RRULE string, DTSTART, and timezone identifier, then expand occurrences on-

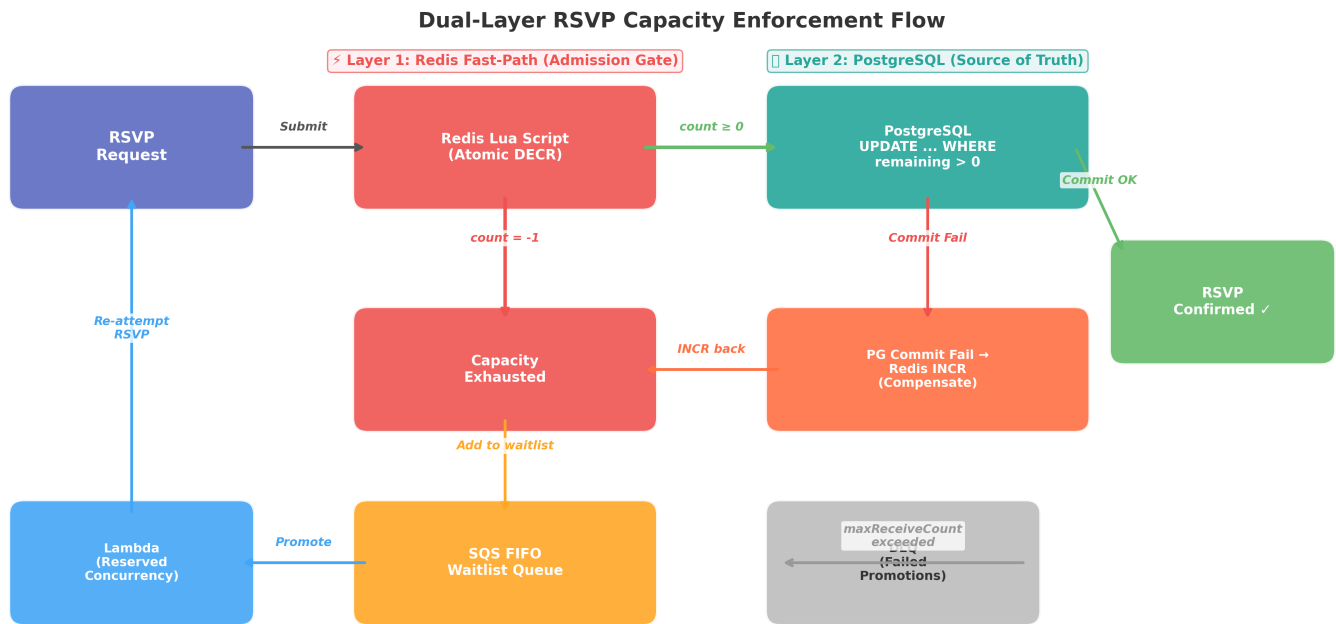


Figure 1: Dual-Layer RSVP Capacity Enforcement Flow

read within a bounded time window rather than pre-generating all instances [27]. PostgreSQL extensions such as `postgres-rrule` support parsing RRULE strings, generating occurrence sequences, and handling EXDATE/RDATE, but critically require bounded rules (UNTIL or COUNT) since infinite recurrence is not supported [13]. ROCA should enforce RRULE validation at the API layer to reject unbounded rules before they reach the database.

Exception Handling and Series Modifications. Single-instance exceptions use EXDATE entries on the master event plus a separate override row linked via `master_event_id` and the original occurrence timestamp. “This-and-future” edits require splitting the series — truncating the original master’s RRULE with an UNTIL at the split point and creating a new master for the modified tail [28]. Two critical failure modes dominate recurring event implementations: treating an exception as a new event (causing duplicate appearances) and applying series edits on top of exceptions (silently reverting user overrides). Sync logic must check for an exception marker before applying either path [27].

DST Handling. A recurring event scheduled for “every Sunday at 09:00” is not the same as “every 7 days from a starting timestamp.” When the UTC offset changes at DST transitions, those two interpretations drift by an hour and the calendar quietly becomes wrong [14]. ROCA must expand occurrences through named IANA timezones (e.g., America/Toronto) rather than fixed UTC offsets to prevent this drift.

Cancellation Cascade Orchestration. Cancelling a recurring event series triggers a multi-step cascade: cancel future instances, update RSVP states, trigger refund processing, and send notifications. The saga pattern implemented through Step Functions provides centralized coordination — each Lambda step publishes an event triggering the next transaction, with compensating transactions to undo changes if any step fails [11]. The orchestrator manages retries, parallel branches, timeouts, and failures across Lambda functions handling discrete tasks [12]. Step Functions standard workflows automatically emit execution status events to the default EventBridge event bus, enabling loosely-coupled downstream services to react to cancellation completions without direct coupling [29]. For RSVP consistency during cascades, the transactional outbox pattern — writing the RSVP state change and an outbox event in the same database transaction, then relaying to EventBridge — prevents race conditions between concurrent series modifications and individual RSVP updates.

6 Performance Efficiency — Event Discovery Search

PostgreSQL 15 Full-Text Search. At ROCA’s current scale, PostgreSQL 15 provides a production-ready search engine without additional infrastructure [30]. The foundation is a pre-computed `tsvector` column with `setweight` for title/description boosting, indexed with a GIN index. Benchmarks demonstrate that replacing

LIKE '%keyword%' queries with a GIN-indexed tsvector column on a 4.2-million-row table reduced query time from several seconds to under 12 milliseconds, with the index adding approximately 380 MB on a 22 GB table [31]. For typo-tolerant fuzzy matching, pg_trgm GIN indexes complement tsvector full-text search [32].

Compound Query Strategy. Event discovery queries typically combine text search, geospatial proximity (PostGIS), temporal filtering, and category facets. Rather than attempting a single composite index, ROCA should use separate specialized indexes — GIN for tsvector, GiST for PostGIS geometry (ST_DWithin), B-tree for timestamp ranges, and GIN for category arrays — since PostgreSQL's query planner performs BitmapAnd across them efficiently [33].

Redis Caching for Search. The cache-aside pattern should be implemented by hashing query parameters (text, geo bounds, date range, categories) into a composite cache key, checking Redis first, and storing results with a TTL on miss [34]. Short TTLs of 30–120 seconds are appropriate for event search results given frequent state changes. Cache entries should be tagged with event IDs so that event updates or cancellations trigger targeted invalidation rather than full cache flushes [35].

OpenSearch Migration Criteria. PostgreSQL FTS handles the vast majority of applications without needing a dedicated search service. Migration to OpenSearch should be considered only when the dataset exceeds single-node capacity (roughly 50–100 million rows), search becomes the core product requiring advanced BM25 tuning or function scoring, horizontal scaling is needed, or complex autocomplete with edge n-grams is required [36]. PostgreSQL offers superior transactional consistency and zero sync lag compared to Elasticsearch, which excels at faceted search and relevance tuning at scale [36].

7 Operational Excellence — State Machines and Observability

Workflow Type Selection. The Event Lifecycle state machine (DRAFT→PUBLISHED→ACTIVE→COMPLETED) should use Step Functions Standard Workflows, which provide exactly-once execution semantics and can run up to one year — ideal for long-running, auditable workflows where non-idempotent actions such as payment processing must not be duplicated [37]. The RSVP state machine (PENDING→CONFIRMED→CHECKED_IN→ATTENDED) should use Express Workflows, which handle high-volume, short-duration tasks at lower cost and are appropriate when all state transitions are idempotent [38]. Express Workflows can be nested within Standard Workflows for selective checkpointing when a single event creation triggers multiple RSVP-related sub-processes [39].

Error Handling and DLQ Integration. The AWS Well-Architected Serverless Applications Lens recommends that Step Functions Retry/Catch replace custom try/catch logic, with failed state runs routing to SQS DLQs for inspection and replay [40]. Each Task state should configure Retry with exponential backoff, and Catch blocks should serve as fallback to route failures to an SQS DLQ or invoke a compensating Lambda [41]. For TIMED_OUT executions in Standard workflows, EventBridge listeners should invoke remediation actions [41].

Cross-Service Decoupling via EventBridge. Step Functions automatically emits execution status change events to the default EventBridge event bus, enabling downstream services (notification via SES/SNS, analytics, audit logging) to react to state transitions without polling [29]. EventBridge rules should be configured with DLQs to capture failed event deliveries with error metadata for debugging [42].

Observability Stack. X-Ray should be enabled on all state machines to visualize components, identify performance bottlenecks, and trace requests end-to-end across Step Functions and downstream services [43]. X-Ray integrates with EventBridge to propagate trace context to downstream targets [44]. CloudWatch should monitor baseline metrics including ExecutionsStarted and ExecutionsTimedOut, with alarms set when performance deviates from established baselines [45].

8 Cost Optimization and Sustainability

Express Workflows for the high-volume RSVP flow provide significant cost savings over Standard Workflows, as they are priced per execution rather than per state transition [38]. PostgreSQL full-text search eliminates the need for a separate OpenSearch cluster at ROCA's current scale, avoiding substantial infrastructure cost while maintaining transactional consistency and zero sync lag [36]. Redis caching of search results and capacity state reduces both Lambda invocations and RDS query load, lowering compute and database costs proportionally to cache hit rates. Lambda reserved concurrency serves a dual purpose: it protects downstream resources like database connections from being overwhelmed [9] while also capping maximum concurrent execution costs during traffic spikes.

For sustainability, the Multi-AZ RDS deployment is already optimized for availability; right-sizing the instance class based on actual query patterns (particularly after implementing Redis caching and GIN index optimizations) can reduce the compute footprint. Infrastructure-as-code practices should be enforced to ensure reproducible, minimal-waste deployments, and automated testing pipelines should validate that Lambda function memory and timeout

configurations are right-sized for actual execution profiles.

9 Prioritized Recommendations and Batch 1 Cross-Reference

The following table consolidates all recommendations across the six Well-Architected pillars, organized by priority tier with explicit cross-references to Batch 1 gaps.

Priority	Timeline	Recommendation	Pillar	Batch 1 Gap?
P0	30 days	Deploy AWS WAF on all API Gateway payment endpoints [3, 4]	Security	Yes — No WAF
P0	30 days	Implement Stripe webhook HMAC-SHA256 signature verification with KMS-stored secrets [21, 22]	Security	New
P0	30 days	Add DLQs to all SQS queues and Step Functions Catch blocks [10, 40]	Reliability	Yes — No DLQ
P0	30 days	Implement Redis Lua + PostgreSQL dual-layer capacity enforcement [6, 7]	Reliability	New
P0	30 days	Enforce SAQ-A script-security eligibility with CSP headers [1, 2]	Security	New
P1	60 days	Migrate event lifecycle to Step Functions Standard Workflows [37]	Operational Excellence	Yes — No Step Functions
P1	60 days	Migrate RSVP transitions to Step Functions Express Workflows [38]	Operational Excellence	Yes — No Step Functions
P1	60 days	Deploy EventBridge custom bus for cross-service event propagation [29]	Operational Excellence	Yes — No EventBridge
P1	60 days	Implement saga pattern for cancellation cascades with compensating transactions [11, 12]	Reliability	New
P1	60 days	Implement SQS FIFO waitlist promotion with Lambda reserved concurrency [9, 10]	Reliability	New
P1	60 days	Implement JWT token rotation mechanism	Security	Yes — No JWT rotation
P2	90 days	Optimize PostgreSQL FTS with GIN-indexed tsvector and pg_trgm [31, 32]	Performance	New
P2	90 days	Implement Redis cache-aside for search with tag-based invalidation [34, 35]	Performance	New
P2	90 days	Enable GuardDuty for continuous threat detection and S3 scanning [5]	Security	Yes — No GuardDuty
P2	90 days	Enable X-Ray tracing across Step Functions and EventBridge [43, 44]	Operational Excellence	New

Priority	Timeline	Recommendation	Pillar	Batch 1 Gap?
P2	90 days	Enforce bounded RRULE validation (UNTIL/COUNT) and IANA timezone expansion [13, 14]	Reliability	New

Table: Prioritized Recommendations Matrix — ROCA Batch 2 Well-Architected Review

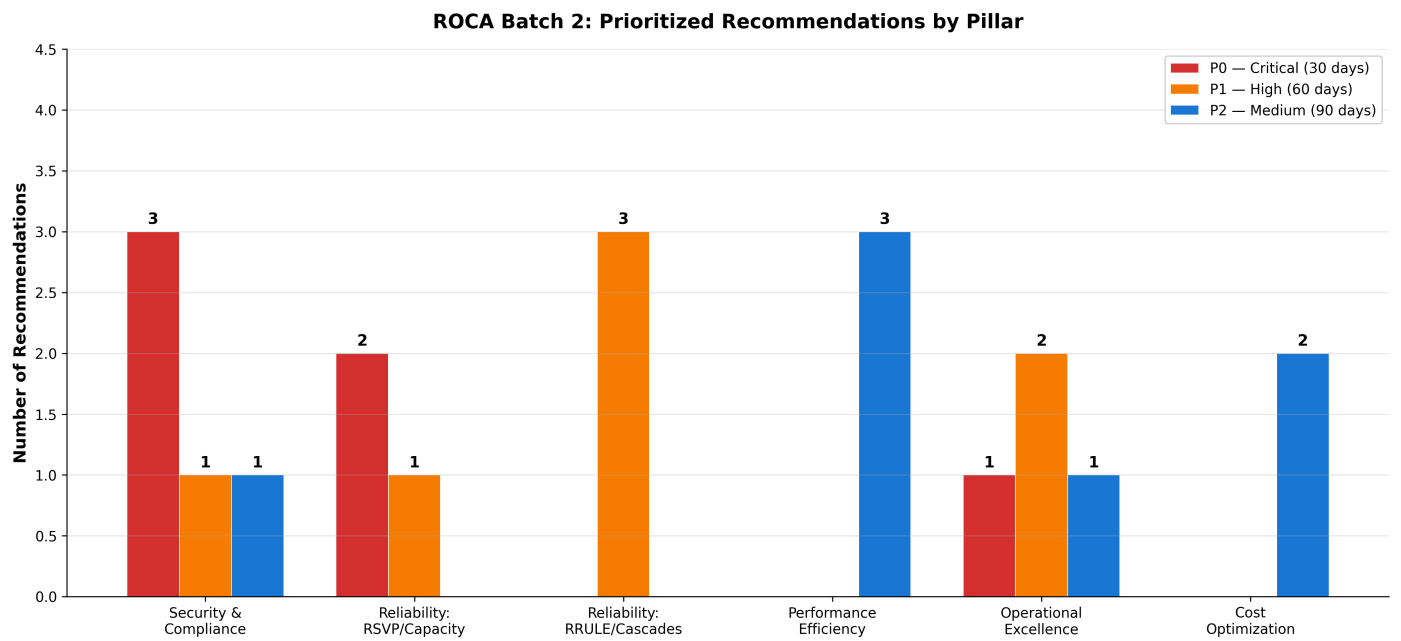


Figure 2: Prioritized Recommendations by Pillar

Chart Explanation: This chart visualizes the distribution of recommendations across the six Well-Architected pillars by priority tier. Security and Reliability dominate the P0 (Critical) tier, reflecting the urgency of WAF deployment for PCI DSS compliance [3, 4] and atomic capacity enforcement for RSVP integrity [6, 7]. Operational Excellence concentrations at P1 reflect the Batch 1 carry-forward gaps in Step Functions [37, 38] and EventBridge [29]. Performance optimizations at P2 reflect that PostgreSQL FTS is adequate at current scale [30] but should be proactively optimized.

Five of the sixteen recommendations directly address Batch 1 gaps (WAF, DLQ, Step Functions, EventBridge, JWT rotation), confirming that these architectural debts compound significantly when payment processing and high-concurrency reservation logic enter scope. The P0 items should be treated as blockers for any production payment processing, while P1 items establish the orchestration foundation needed for Batch 3 and Batch 4 flows.

10 References

- 1. <https://cmmc.securitymetrics.com/blog/big-changes-for-saq-a>
- 2. <https://www.akamai.com/content/akamai/en/blog/security/2025/feb/pci-dss-v4-0-1-changes-qualify-saq-a>
- 3. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
- 4. <https://aws.amazon.com/about-aws/whats-new/2025/03/aws-waf-pci-compliance-protection-partner-solutions/>
- 5. https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_settingup.html
- 6. <https://redis.io/docs/latest/develop/programmability/eval-intro/>
- 7. <https://www.postgresql.org/docs/current/explicit-locking.html>
- 8. <https://docs.aws.amazon.com/whitepapers/latest/database-caching-strategies-using-redis/caching-patterns.html>
- 9. <https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>

10. [https:// docs.aws.amazon.com/ AWSSimpleQueueService/ latest/ SQSDeveloperGuide/ sqs-dead-letter-queues.html](https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html)
11. <https:// docs.aws.amazon.com/ prescriptive-guidance/ latest/ patterns/ implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
12. <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/saga-orchestration.html>
13. <https://github.com/volkanunsal/postgres-rrule>
14. <https://appmaster.io/blog/postgresql-recurring-schedules-time-zones>
15. AWS Well-Architected Framework Review of ROCA New Member Onboarding Flow
16. <https://stripe.com/guides/pci-compliance>
17. <https://blog.pcisecuritystandards.org/faq-clarifies-new-saq-a-eligibility-criteria-for-e-commerce-merchants>
18. <https:// docs.aws.amazon.com/ prescriptive-guidance/ latest/ aws-security-reference-architecture-pci-data-security-standard-dss/introduction.html>
19. <https://docs.aws.amazon.com/solutions/tokenization-to-improve-data-security-and-reduce-audit-scope-on-aws/>
20. <https://aws.amazon.com/compliance/services-in-scope/PCI/>
21. <https://docs.stripe.com/webhooks/signature>
22. <https://git.durrantlab.pitt.edu/LuciferForge/stripe-webhook-handler>
23. <https://www.pcicompliance.com/pci-serverless/>
24. <https://redis.io/tutorials/inventory-reservation-in-real-time-with-redis/>
25. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/wait-event.locktuple.html>
26. <https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html>
27. <https://cli.nylas.com/guides/recurring-calendar-events-api>
28. <https://developer.nylas.com/docs/v3/calendar/recurring-events/>
29. <https://docs.aws.amazon.com/step-functions/latest/dg/eventbridge-integration.html>
30. <https://tutorials.technology/tutorials/postgresql-full-text-search-tutorial-2026.html>
31. <https://www.jusdb.com/blog/postgresql-full-text-search-tsvector-gin-index>
32. <https://tacnode.io/post/full-text-search-postgresql-complete-guide>
33. <https://www.postgresql.org/docs/current/gin.html>
34. <https://redis.io/tutorials/howtos/solutions/microservices/caching/>
35. Source: wikipedia.org
36. <https://www.kindatechnical.com/postgresql/postgresql-vs-elasticsearch-when-to-use-each.html>
37. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html>
38. <https://docs.aws.amazon.com/step-functions/latest/dg/cost-opt-exp-workflows.html>
39. <https://docs.aws.amazon.com/step-functions/latest/dg/bp-express.html>
40. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/failure-management.html>
41. <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
42. <https://aws.amazon.com/blogs/compute/testing-amazon-eventbridge-events-using-aws-step-functions/>
43. https://docs.aws.amazon.com/en_us/step-functions/latest/dg/concepts-xray-tracing.html
44. <https://docs.aws.amazon.com/xray/latest/devguide/xray-services-eventbridge.html>
45. <https://docs.aws.amazon.com/step-functions/latest/dg/procedure-cw-metrics.html>